西南交通大学
Southwest Jiaotong University

Operating System



Xiangyi Chen
xiangyichen@swjtu.edu.cn

Reference

- Operating Systems— Internals and Design Principles (9th Edition), William Stallings，电子工业出版社
- Operating System Concepts , Abraham Silberschatz, 机械工业出版社
- 陈向群、陈渝译，操作系统：精髓与设计原理（第八版)，电子工业出版社
- 汤小丹，梁红兵，哲凤屏，汤子瀛，现代操作系统，电子工业出版社
- 庞丽萍，郑然，操作系统原理与Linux系统实验，机械工业出版社

Evaluation

西南交通大學
Southwest Jiaotong University

- In-class participation 5%
- Assignment 10%
- Final exam 50%
- Mid-term exam 10%
- Laboratory 25%

Course Outline

- **OS Introduction**
- **Process Management**
  - Process and Threads
  - Scheduling
  - Concurrency
- **Memory Management**
- **Device Management**
- **File Management**

Chapter Outline

- Operating System Objectives
- Evolution of Operating Systems
- Operating System Structures

1. OS Objectives 目标

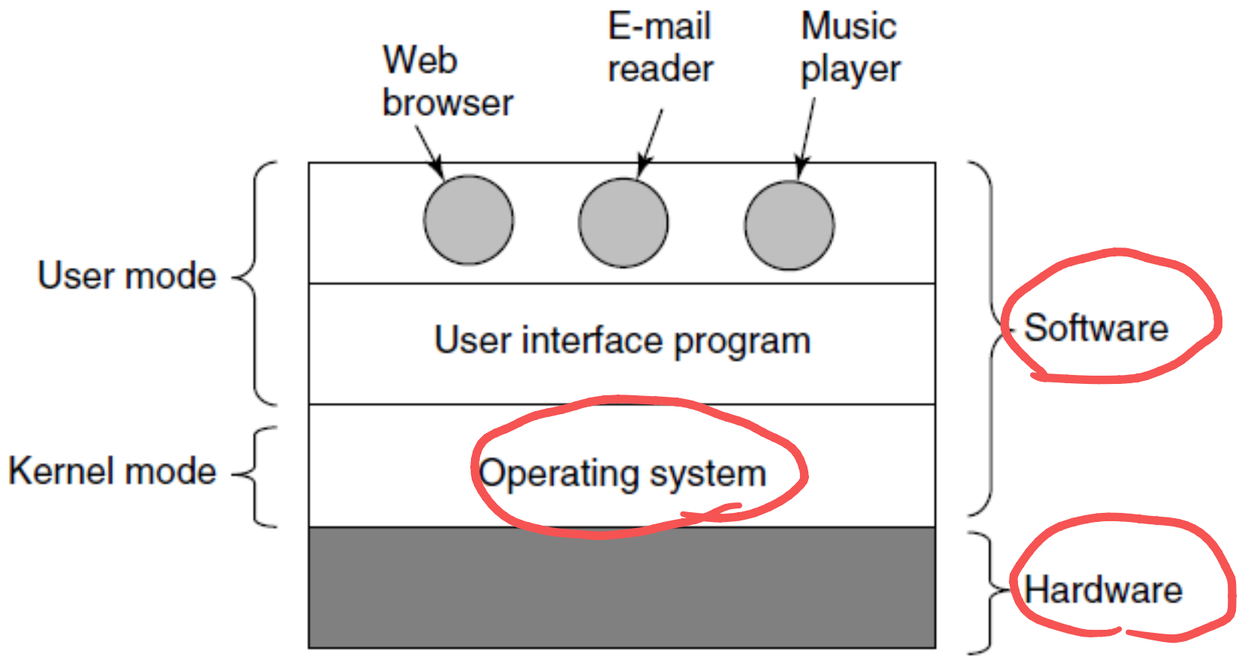
- A program that controls the execution of application programs
- An interface between applications and hardware

OS是控制应用执行的程序，是硬件和软件的接口

Main objectives of an OS

- **Convenience** 方便
- Efficiency 高效
- Ability to evolve 可发展

Components of a Modern Computer



Computer Hardware and Software Infrastructure

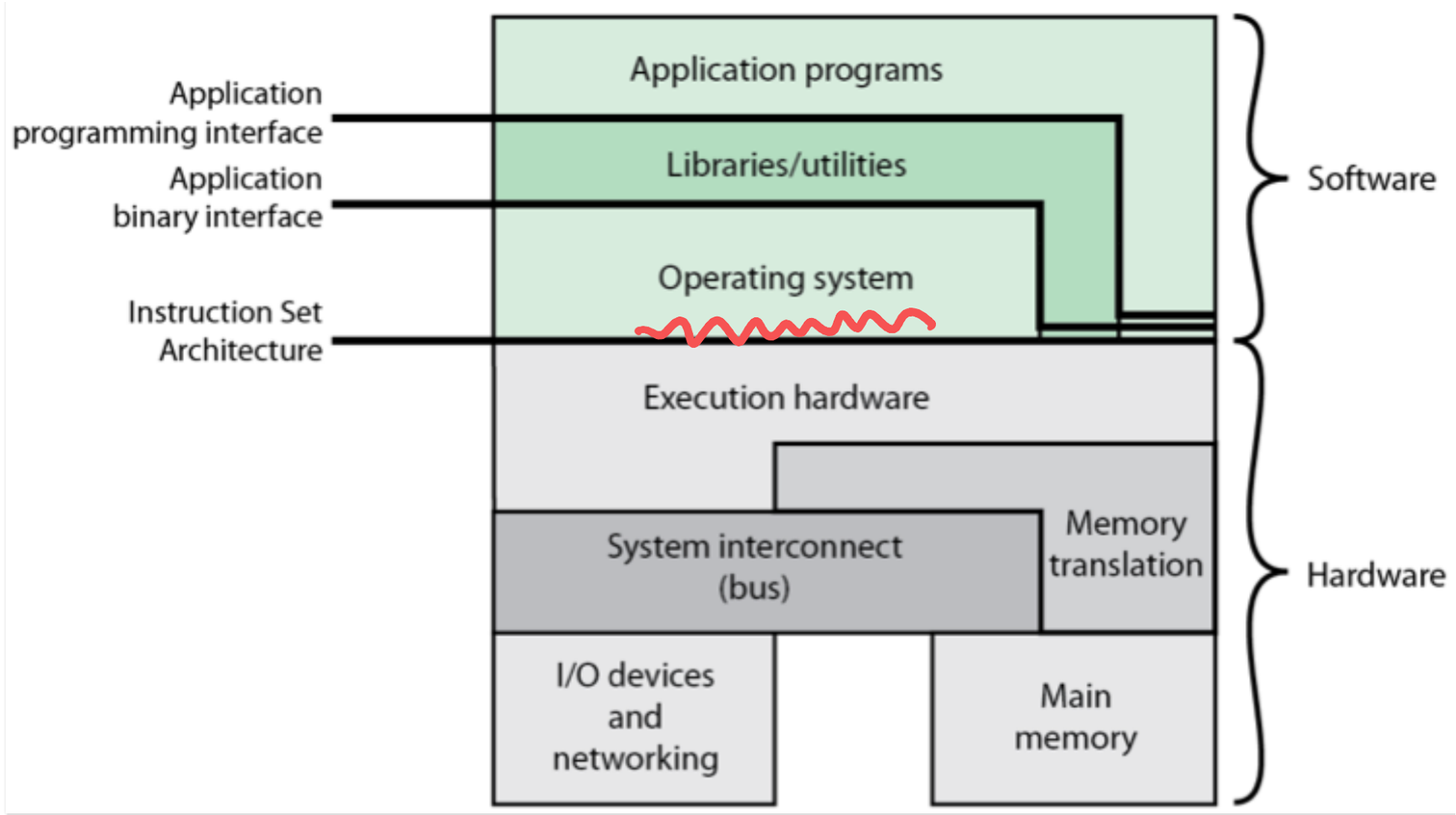


Figure 2.1 Computer Hardware and Software Infrastructure

Components of a Modern Computer

- One or more processors
- Main memory
- Disks
- Printers
- Keyboard
- Mouse
- Display
- Network interfaces
- I/O devices

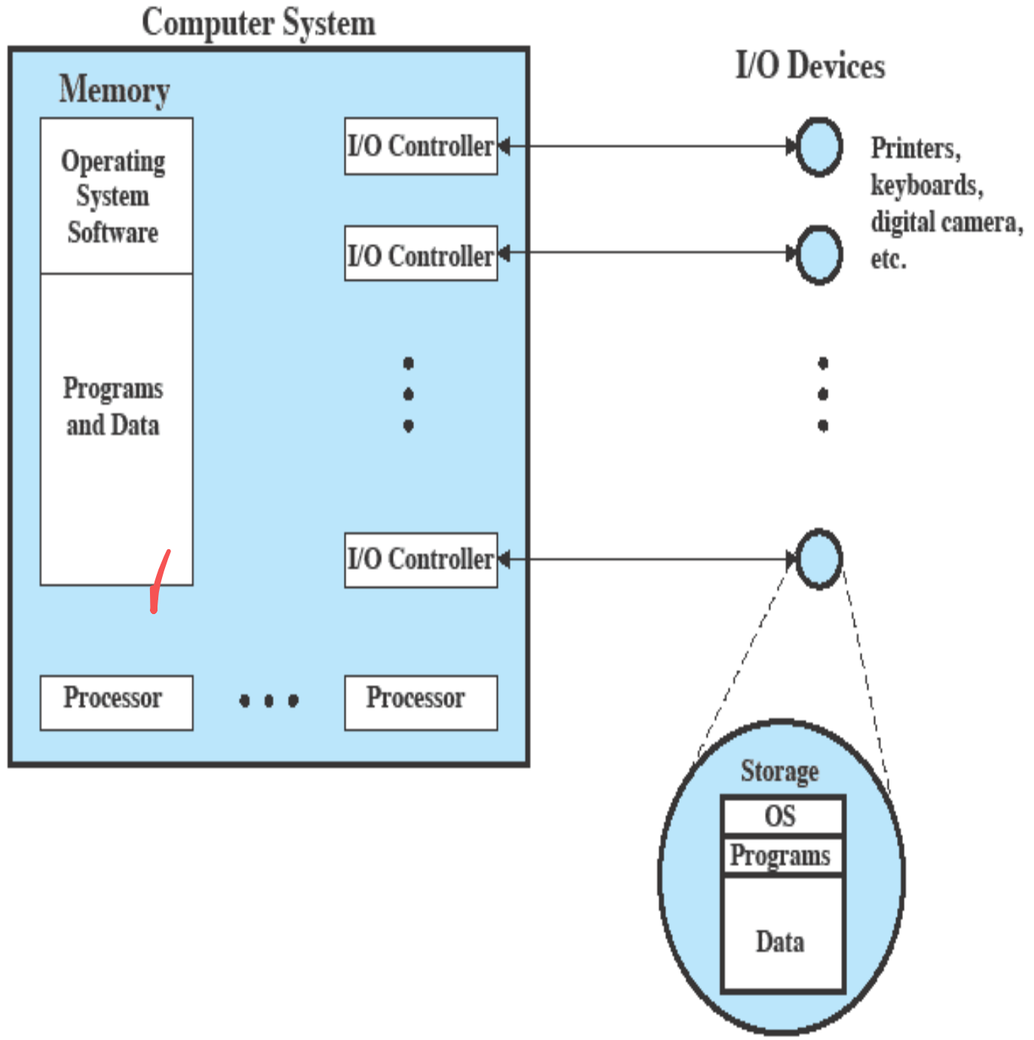


Figure 2.2 The Operating System as Resource Manager

OS Objectives

- A software functions in the same way as ordinary computer software
- The OS relinquishes control for the processor to do some "useful" work and then resumes control

操作系统作为软件，其运行方式与其他普通计算机软件一样。

将处理器的控制权移交，使其执行各类实际业务操作。随后重新收回控制权

Main objectives of an OS

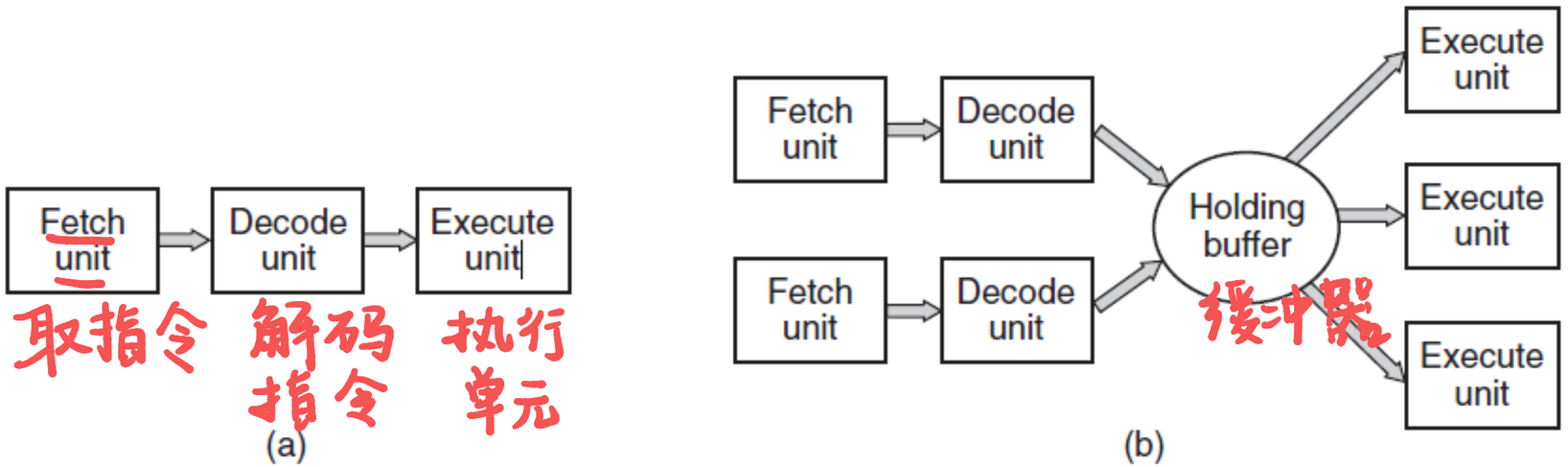
- Convenience
- **Efficiency**
- Ability to evolve

OS as a Resource Manager

作为资源管理器的操作系统

- **Top down view** 自上而下视角
  - Provide abstractions to application programs 为应用程序提供抽象接口
- **Bottom up view** 自下而上
  - Manage pieces of complex system 管理复杂系统的各类资源
- **Alternative view**
  - Provide orderly, controlled allocation of resources 分配

Computer Hardware is not simple



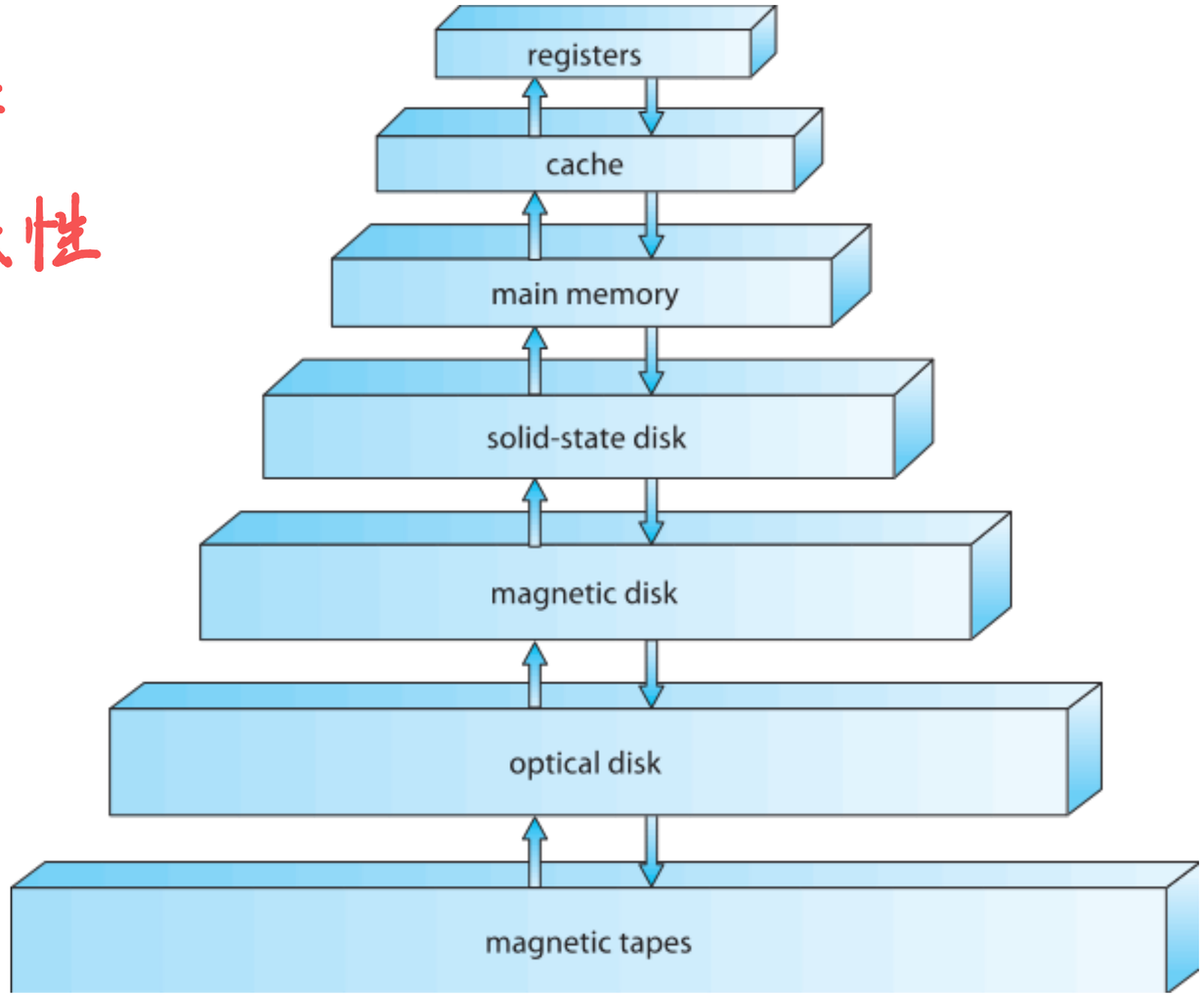
(a) A three-stage pipeline. (b) A superscalar CPU.

三级流水线

The Memory/Storage Hierarchy

层次.

- Speed 速度
- Cost 成本
- Volatility 易失性

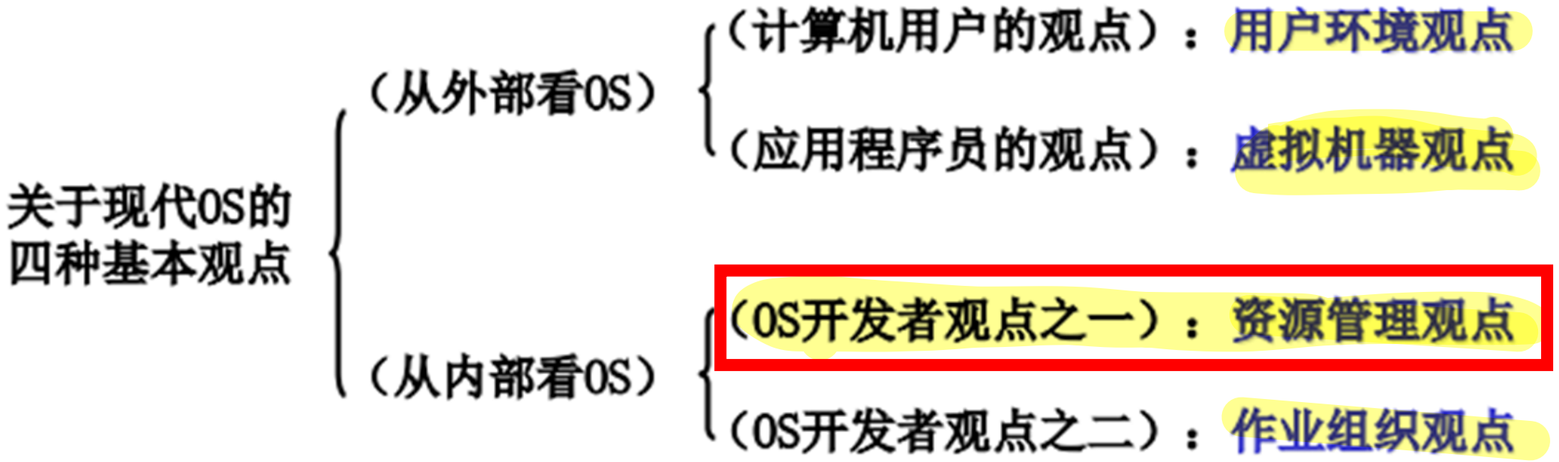


OS's To-Do List 核心工作

- Start and stop programs 启动与终止程序
- Manage memory 管理内存
- Handle input/output 处理输入/输出操作
- Overlap I/O with processing 输入/出与运算的并发运行
- Handle file systems 管理文件系统
- Multiple program running (concurrency) 并发运行多项目
- Networking 网络通信管理
- Security 安全
- Error handling and recovery 错误处理与恢复
- Act as “computer manufacturer’s GUI”

计算机厂商的GUI

What is OS


关于现代OS的
四种基本观点
（从外部看OS）
（计算机用户的观点）：用户环境观点
（应用程序员的观点）：虚拟机器观点
（从内部看OS）
（OS开发者观点之一）：资源管理观点
（OS开发者观点之二）：作业组织观点

OS Objectives

西南交通大学
Southwest Jiaotong University

- **A major OS will evolve over time for a number of reasons:**
  - Hardware upgrades
  - New types of hardware
  - New services 新的服务需求
  - Fixes

Main objectives of an OS
☐ Convenience ☐ Efficiency ☐ **Ability to evolve**

2. Evolution of Operating System

西南交通大学
Southwest Jiaotong University

- Serial Processing 串行
- Simple Batch Systems 简单批处理
- Multiprogrammed Batch Systems 多道批处理
- Time Sharing Systems 分时系统
- Modern Operating Systems 现代操作系统

Serial Processing（串行）

西南交通大学 Southwest Jiaotong University

- No operating system
- Machines run from a console with display lights and toggle switches, input device, and printer.
- Two main problems
  - **Scheduling**: waste processing time.
    - **Setup time**: Setup included loading the compiler, source program, saving compiled program, and loading and linking

处理器时间存在大量浪费

调度

准备时间

Simple Batch Systems（批处理）

- **Monitors**（监督程序）:**Software that controls the running programs**
  - **Resident monitor** is in main memory and available for execution
- **Monitor batches jobs together**
- **Monitor reads in job and gives control**
- **Job returns control to monitor**

常驻监督程序驻留在主存储器中

可随时执行

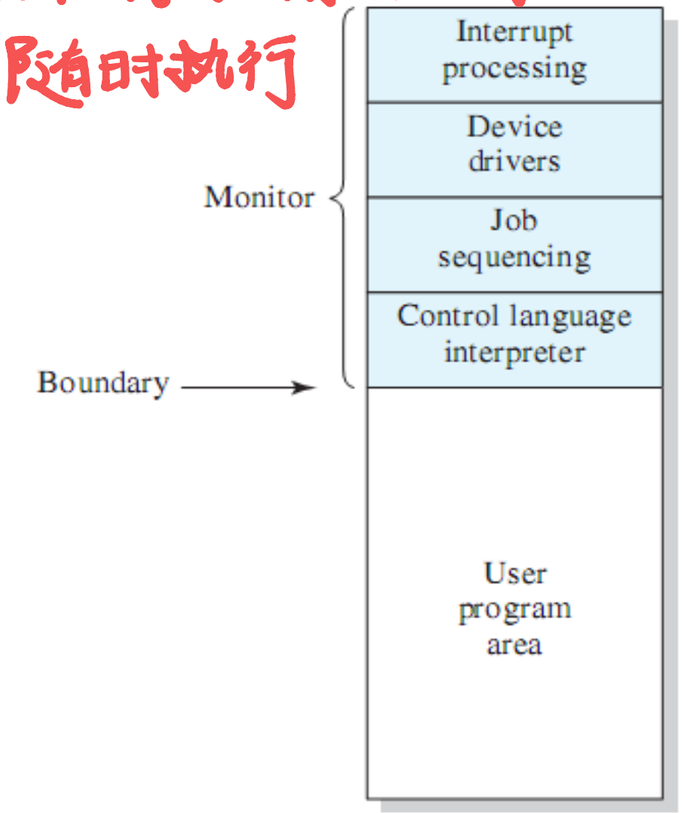


Figure 2.3 Memory Layout for a Resident Monitor

Simple Batch System Overhead

西南交通大学 Southwest Jiaotong University

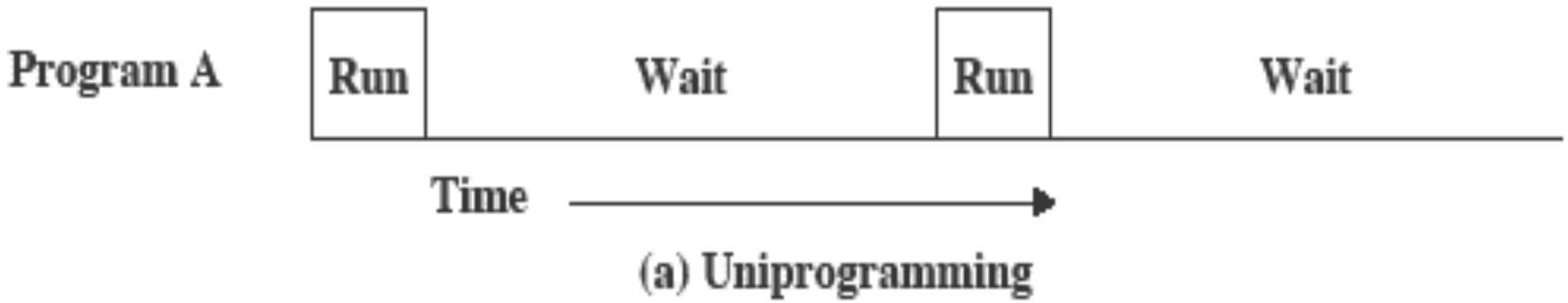
CPU

- **Processor time alternates between execution of user programs and execution of the monitor**
- **Overhead sacrifices:** 在用户程序和监督程序中轮换
  - some main memory is now given over to the monitor
  - some processor time is consumed by the monitor
- **Despite overhead, the simple batch system improves utilization of the computer**

开销

Uniprogramming（单道）

- Processor must wait for I/O instruction to complete before proceeding.
- Be inefficient.
- Simple 单道



(a) Uniprogramming

Simple Batch System Example

空闲状态

- Processor is often idle even with automatic job sequencing
- I/O devices are slow compared to processor

Read one record from file	15 μs
Execute 100 instructions	1 μs
Write one record to file	15 μs
TOTAL	31 μs

Percent CPU Utilization $= \frac{1}{31} = 0.032 = 3.2\%$

Figure 2.4 System Utilization Example

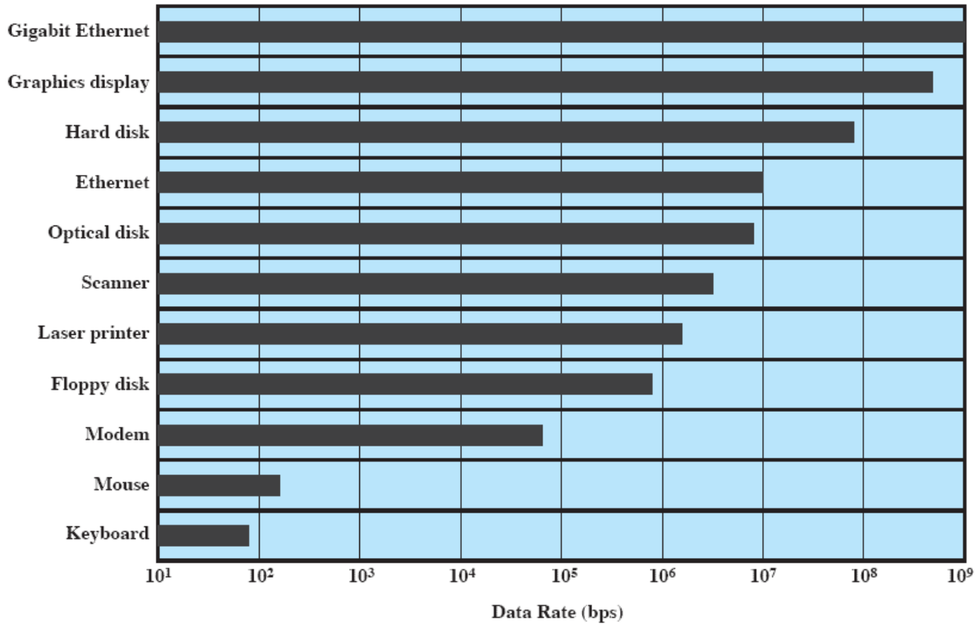
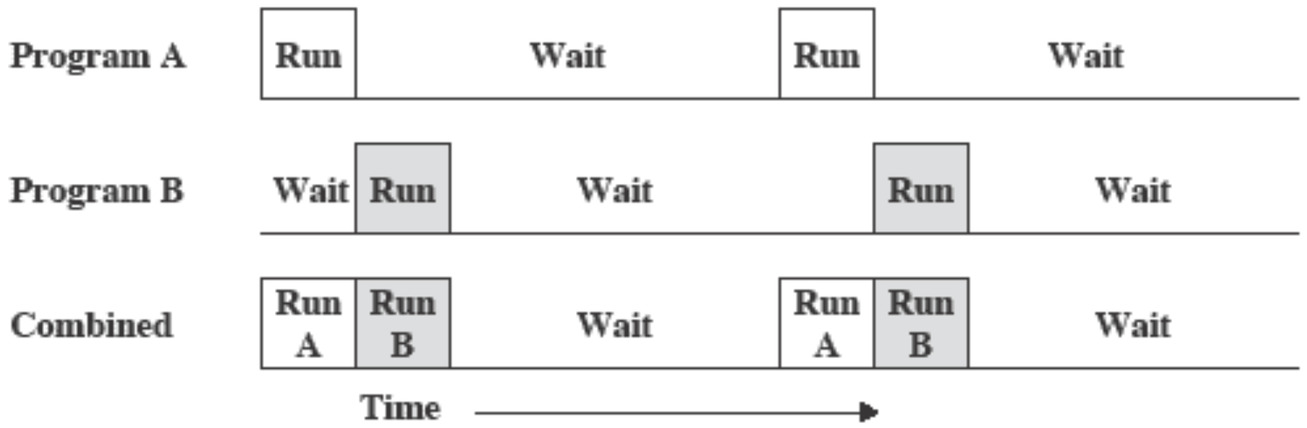


Figure 11.1 Typical I/O Device Data Rates

Multiprogramming（多道）

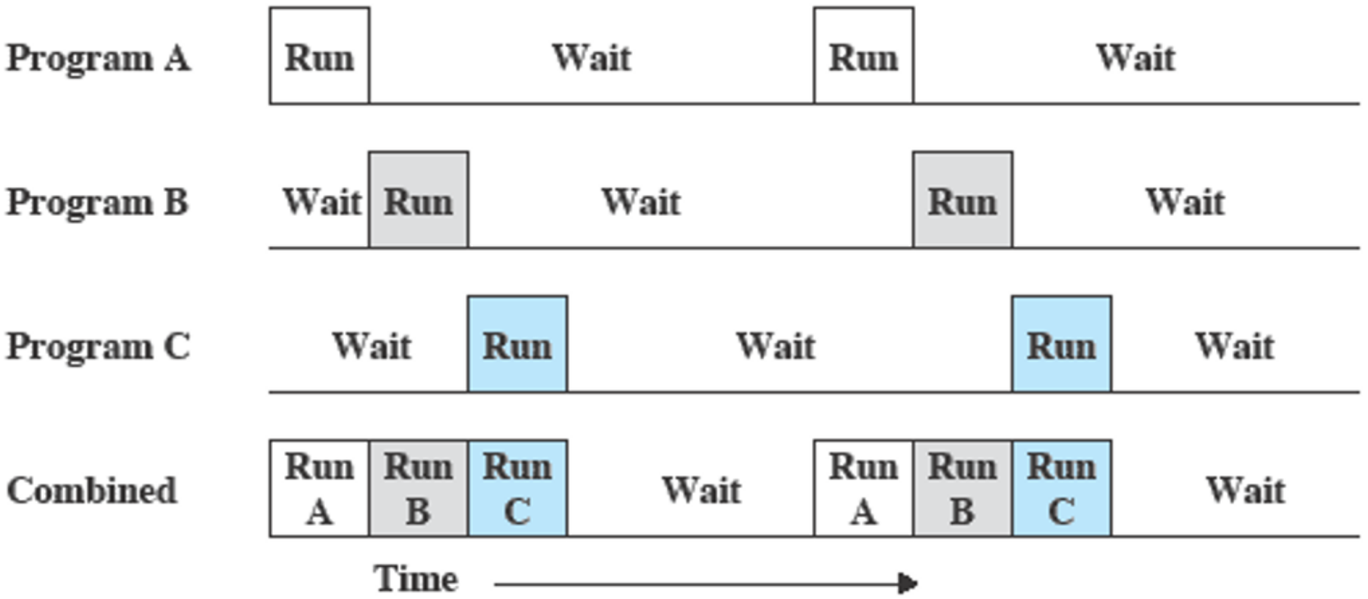
- There must be enough memory to hold the OS (resident monitor) and one user program
- When one job needs to wait for I/O, the processor can switch to the other job, which is likely not waiting for I/O



(b) Multiprogramming with two programs

Multiprogramming

- Multiprogramming also known as multitasking
- Memory is expanded to hold three, four, or more programs and switch among all of them



Multiprogramming Example

西南交通大学 Southwest Jiaotong University

Table 2.1 Sample Program Execution Attributes

	JOB1	JOB2	JOB3
Type of job	Heavy compute	Heavy I/O	Heavy I/O
Duration	5 min	15 min	10 min
Memory required	50 M	100 M	75 M
Need disk?	No	No	Yes
Need terminal?	No	Yes	No
Need printer?	No	No	Yes

Multiprogramming Example

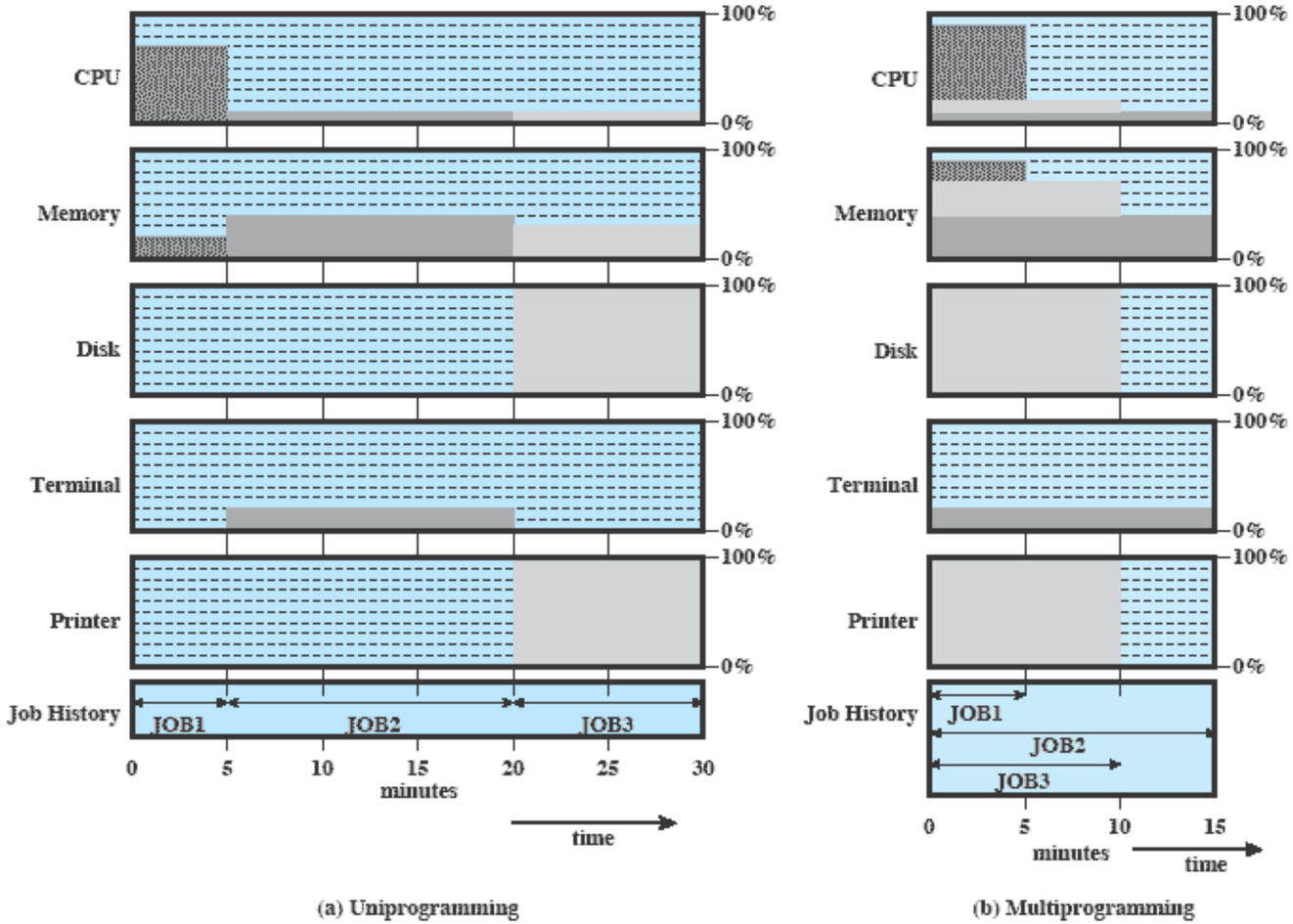


Figure 2.6 Utilization Histograms

Multiprogramming Example

西南交通大学 Southwest Jiaotong University

	Uniprogramming	Multiprogramming
Processor use	22%	43%
Memory use	30%	67%
Disk use	33%	67%
Printer use	33%	67%
Elapsed time	30 min.	15 min.
Throughput rate	6 jobs/hr	12 jobs/hr
Mean response time	18 min.	10 min.

Difficulties with Multiprogramming

西南交通大学 Southwest Jiaotong University

- **Improper synchronization**（同步） 同步异常
  - ensure a process waiting for an I/O device receives the signal
- **Failed mutual exclusion**（互斥） 互斥失效
- **Nondeterminate program operation** （不确定性）
  - program should only depend on input to it
- **Deadlocks**（死锁）

多个进程相互等待而陷入永久阻塞

Time-Sharing Systems （分时）

西南交通大学
Southwest Jiaotong University

- Using multiprogramming to handle multiple interactive （交互） jobs
- Processor's time is shared among multiple users
- Multiple users simultaneously（同时） access the system through terminals

Multiprogramming vs. Time Sharing

	Batch Multiprogramming	Time Sharing
Principal objective（主要目标）	Maximize processor use	Minimize response time
Source of directives to operating system	Job control language commands provided with the job	Commands entered at the terminal

Evolution of Operating System

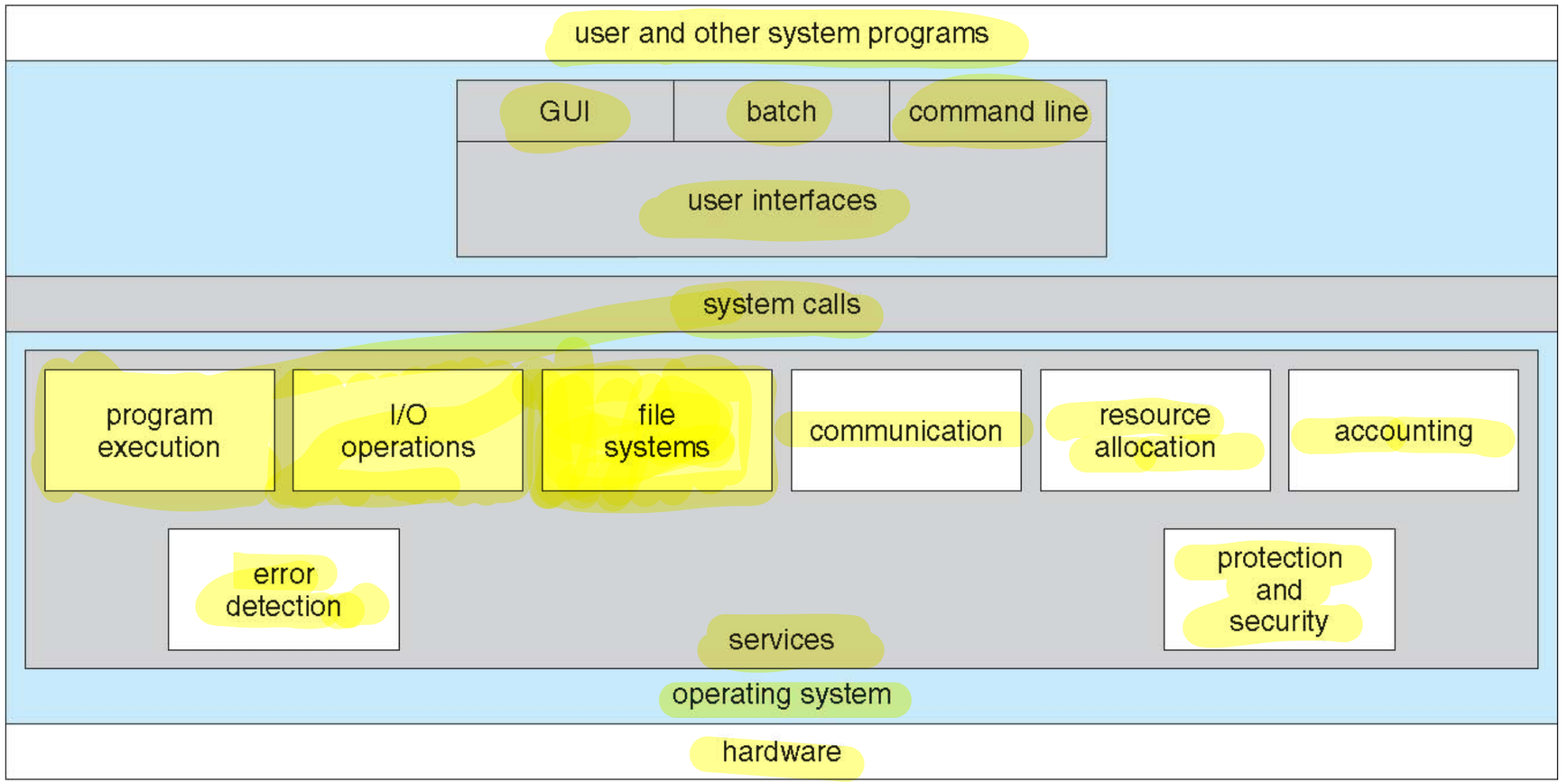
西南交通大学
Southwest Jiaotong University

Short but rich history, 1955-present

- Mainframe Operating Systems
- Personal Computer Operating Systems
- Embedded Operating Systems
- Mobile Operating Systems
- Multiprocessor Operating Systems
- Distributed Operating Systems
- Cloud Operating Systems
- Edge Operating Systems

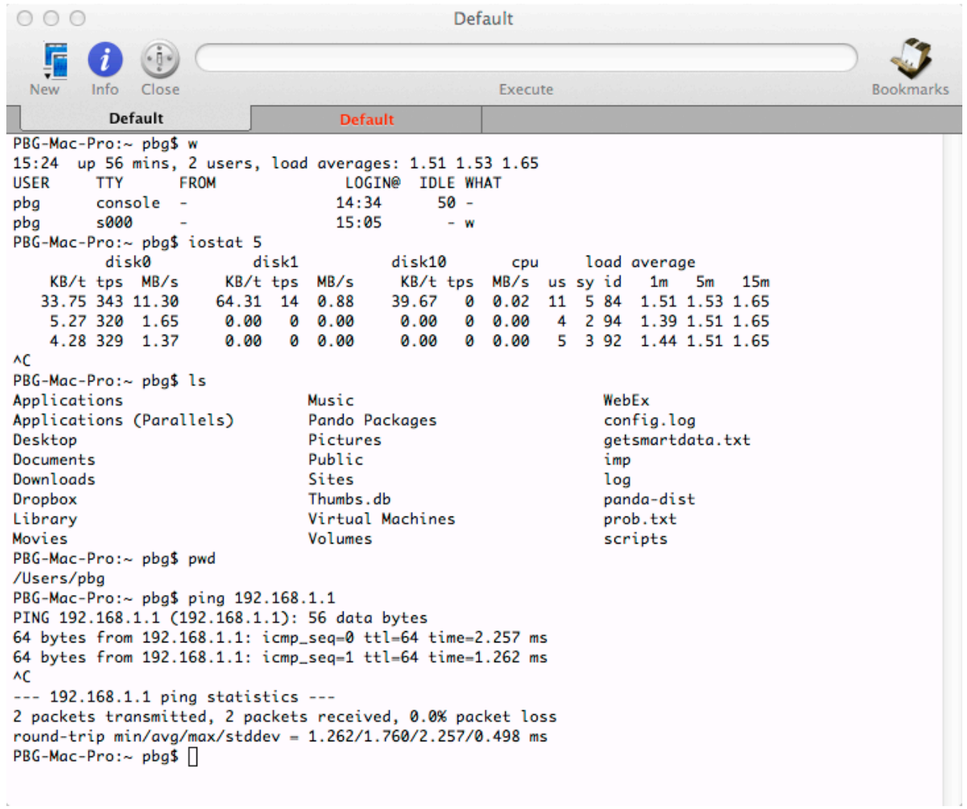
3. Operating System Services

西南交通大学
Southwest Jiaotong University



Operating System Services

- **Some services are helpful to the user:**
  - **User interface** - Almost all operating systems have a user interface (UI), eg. Command-Line (CLI), Graphics User Interface (GUI), Batch





Operating System Services

西南交通大学 Southwest Jiaotong University

- **Operating systems provide an environment for execution of programs**
  - **Program execution -** The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
  - **I/O operations -** A running program may require I/O, which may involve a file or an I/O device
  - **File-system manipulation -** Programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.
  - **Communications** – Processes may exchange information, on the same computer or between computers over a network

Operating System Services (Cont.)

- **Another set of OS functions exists for ensuring the efficient operation of the system itself via resource sharing**
  - **Resource allocation** - When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
  - **Accounting** - To keep track of which users use how much and what kinds of computer resources
  - **Protection and security** - The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other

资源分配 计数

并发性不能相互干扰

System Calls

System Calls

- OS provides programming interface to services
- It is how a program requests a service from an OS's kernel
- May include hardware-related services (for example, accessing a hard disk drive), creation and execution of new processes, and communication with integral kernel services such as process scheduling

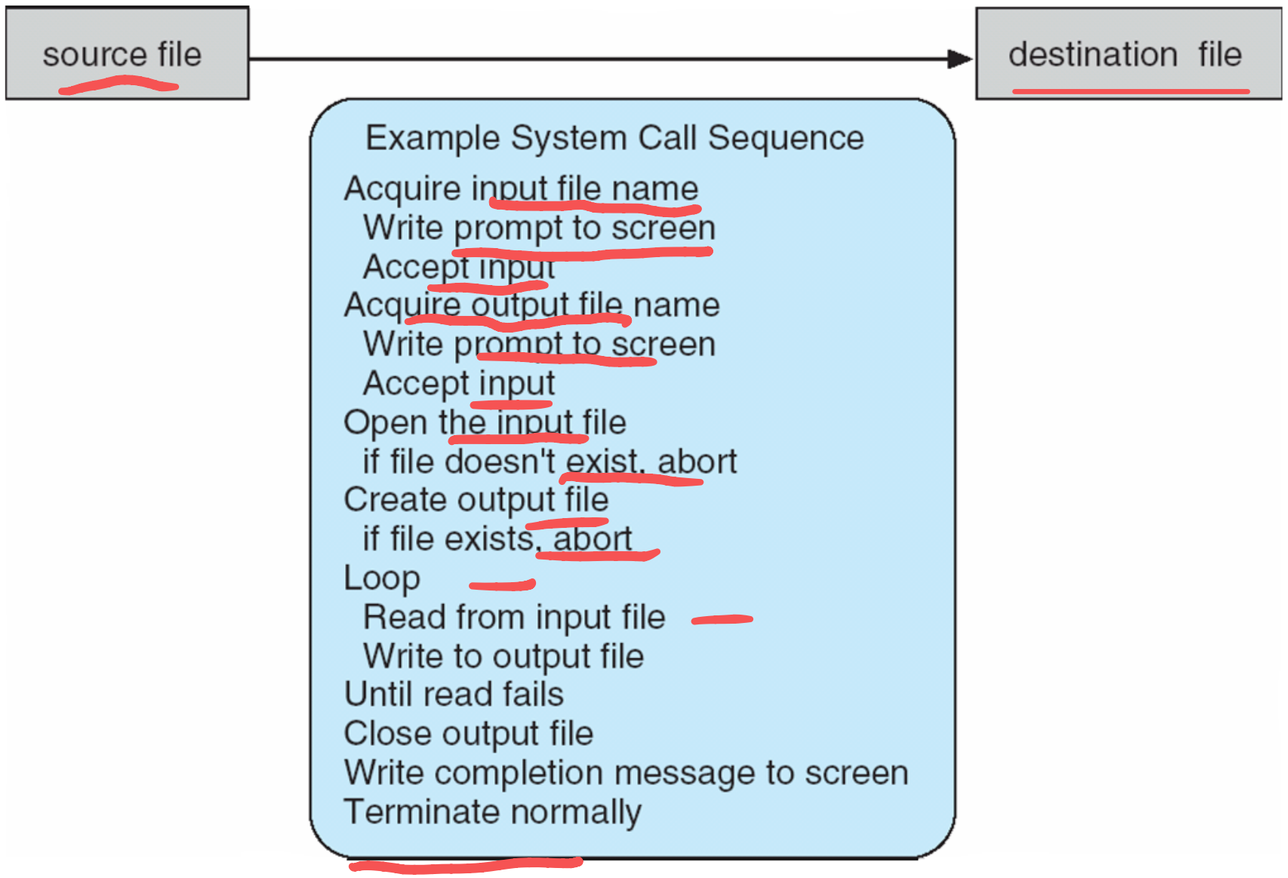
Application Programming Interface (API)

- High-level - usually used by programs
- Rather than direct use of system calls
- Usually less detailed then raw calls

Example of System Calls

西南交通大学
Southwest Jiaotong University

- System call sequence to copy the contents of one file to another file



Example of Standard API

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t    read(int fd, void *buf, size_t count)
```

return value — function name — parameters

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

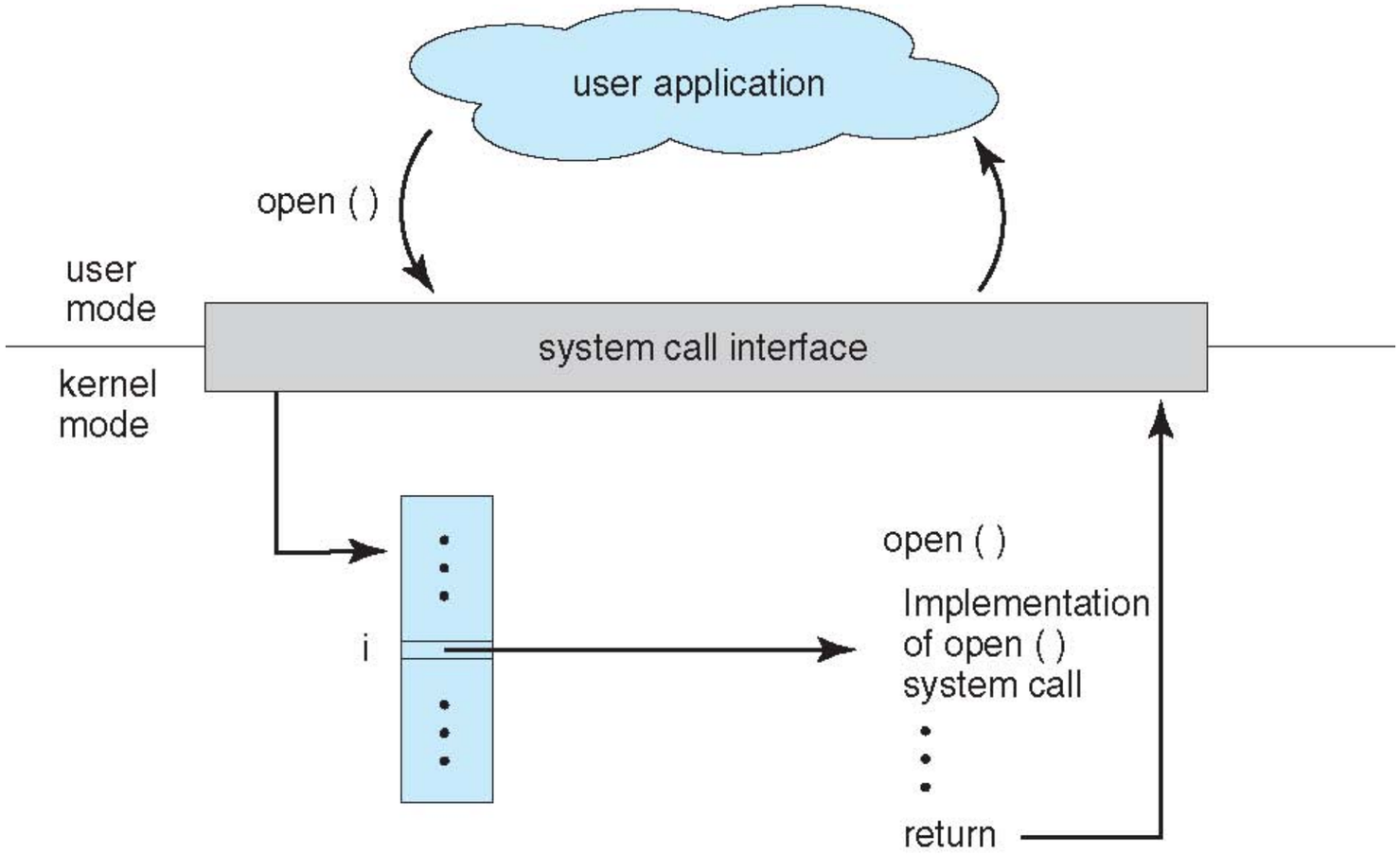
On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns −1.

System Call Implementation

西南交通大学 Southwest Jiaotong University

- **Typically, a number associated with each system call**
  - System-call interface maintains a table indexed according to these numbers
- **The system call interface invokes intended system call in OS kernel and returns status of the system call and any return values**
- **The caller need know nothing about how the system call is implemented**
  - Just needs to obey API and understand what OS will do as a result call
  - Most details of OS interface hidden from programmer by API
    - Managed by run-time support library (set of functions built into libraries included with compiler)

API – System Call – OS Relationship



Types of System Calls

西南交通大学
Southwest Jiaotong University

- **Process control**
  - create process, terminate process
- **File management**
  - create file, delete file, open, close file
- **Device management**
  - request device, release device
- **Information maintenance**
  - get time or date, set time or date
- **Communications**
  - create, delete communication connection
- **Protection**
  - Control access to resources

Examples of Windows and Unix System Calls



西南交通大学
Southwest Jiaotong University

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitlializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Operating System Design and Implementation

西南交通大学
Southwest Jiaotong University

- **Specifying and designing OS is highly creative task of software engineering**
- **User goals**
  - convenient to use,
  - easy to learn,
  - reliable,
  - Safe
  - Fast
- **System goals**
  - easy to design, implement, and maintain
  - flexible,
  - reliable,
  - error-free,
  - efficient

Operating System Structure

西南交通大学
Southwest Jiaotong University

- **General-purpose OS is very large program**
  - Simple Structure
  - Multi-Layer/ Multi-Level
  - Microkernel
  - Modules
  - Hybrid Systems

混合体系结构

OS Kernel

- The OS kernel is a computer program that constitutes the central core of a computer's operating system.
- It has complete control over everything that occurs in the system. As such, it is the first program loaded on startup, and then manages the remainder of the startup, as well as I/O requests from software, translating them into data processing instructions for the CPU.
- It is also responsible for managing memory, and for managing and communicating with external devices, like printers, speakers, etc. The kernel is a fundamental part of a modern computer's operating system.

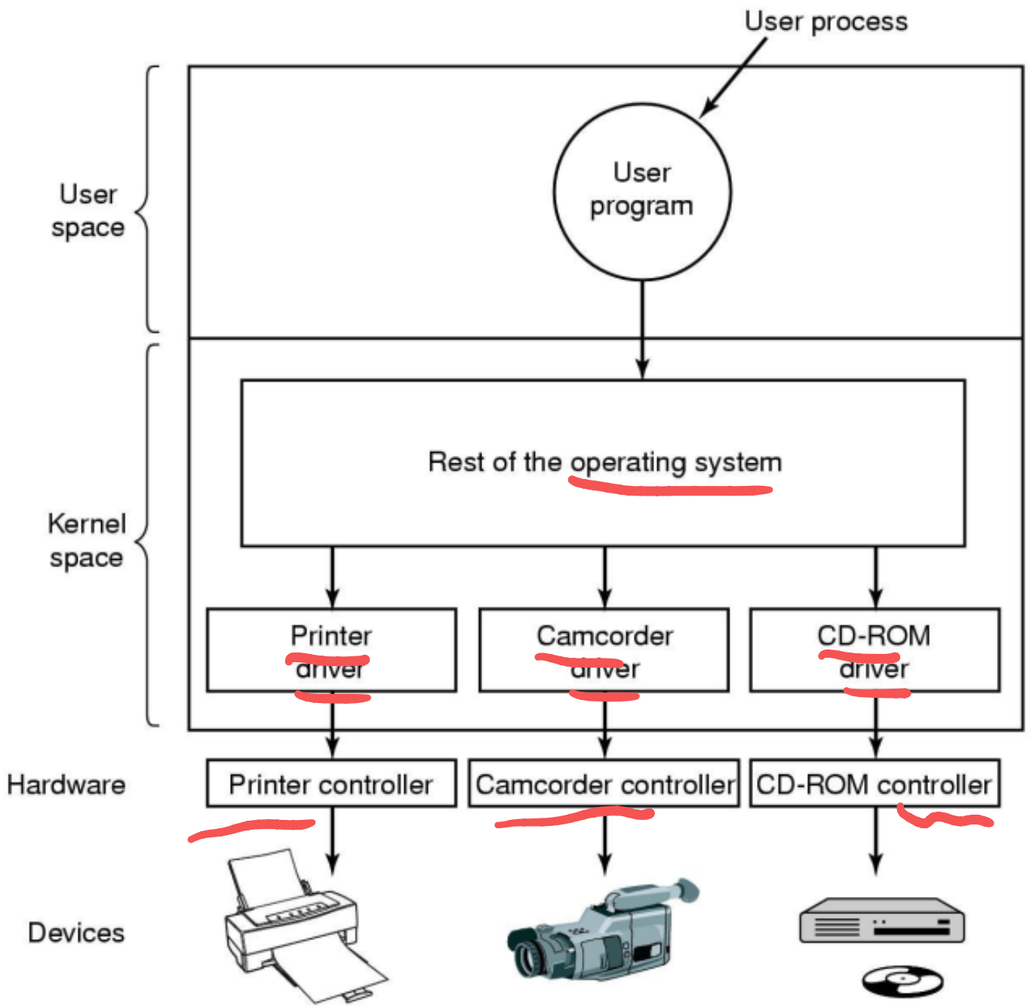
Kernel Space vs. User Space

西南交通大学 Southwest Jiaotong University

- The critical code of the kernel is usually loaded into a protected area of memory, which prevents it from being overwritten by other, less frequently used parts of the OS or by applications.
- The kernel performs its tasks, such as executing processes and handling interrupts, in kernel space, whereas everything a user normally does, such as writing text in a text editor or running programs in a GUI, is done in user space.
- This separation prevents user data and kernel data from interfering with each other and thereby reducing performance or causing the system to become unstable (and possibly crashing).

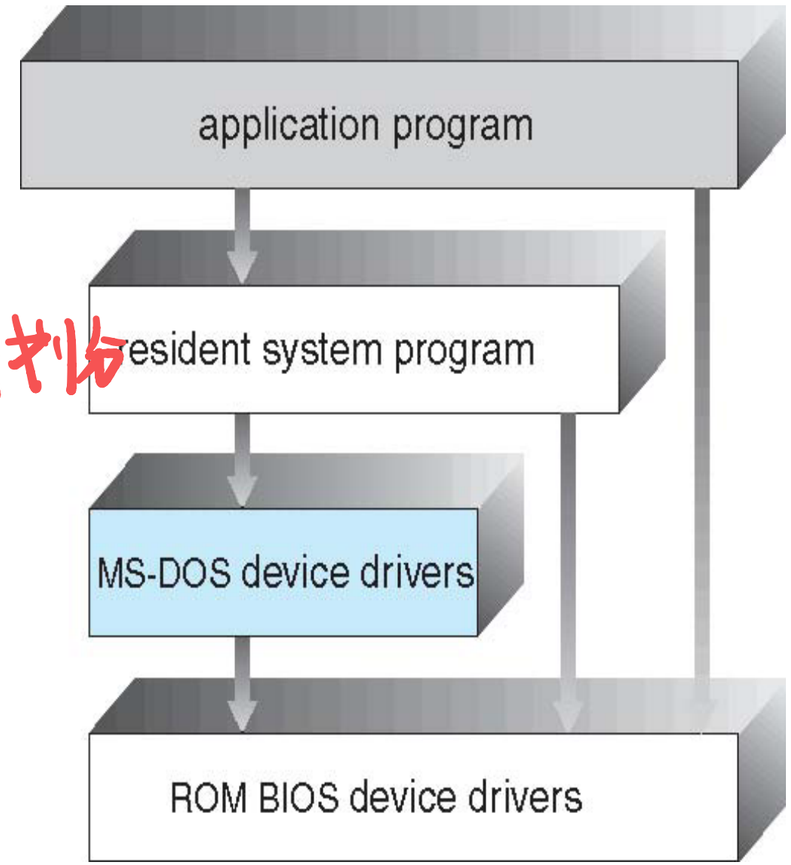
Kernel Space vs. User Space

西南交通大学
Southwest Jiaotong University



Simple Structure

- I.e. **MS-DOS – written to provide the most functionality in the least space**
  - Not divided into modules
  - Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated



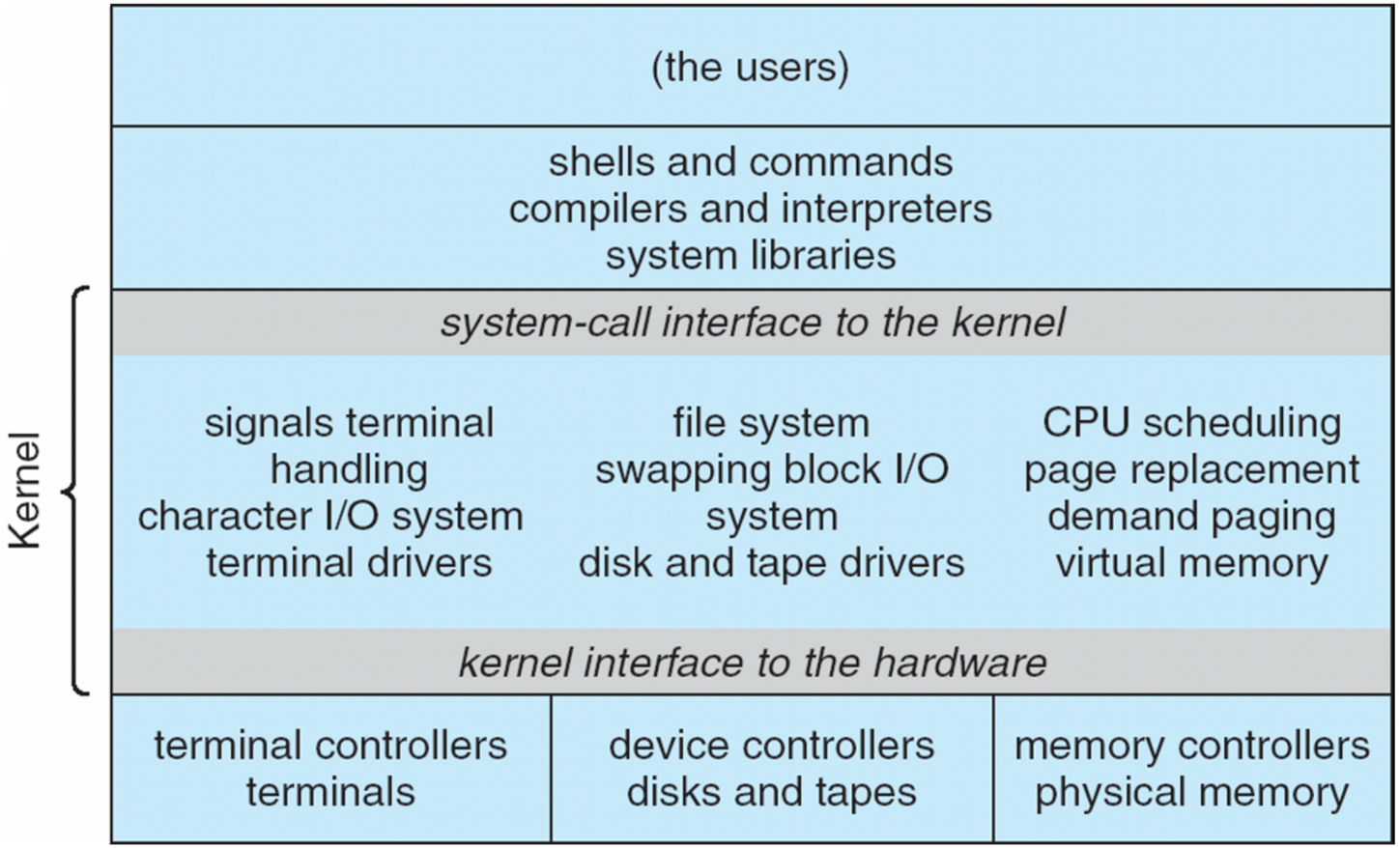
UNIX

- **UNIX – limited by hardware functionality, the original UNIX operating system had limited structuring. The UNIX OS consists of two separable parts**
  - Systems programs (系统程序)
  - The kernel
    - Consists of everything below the system-call interface and above the physical hardware
    - Provides the file system, CPU scheduling, memory management, and other operating-system functions; a large number of functions for one level

Traditional UNIX System Structure

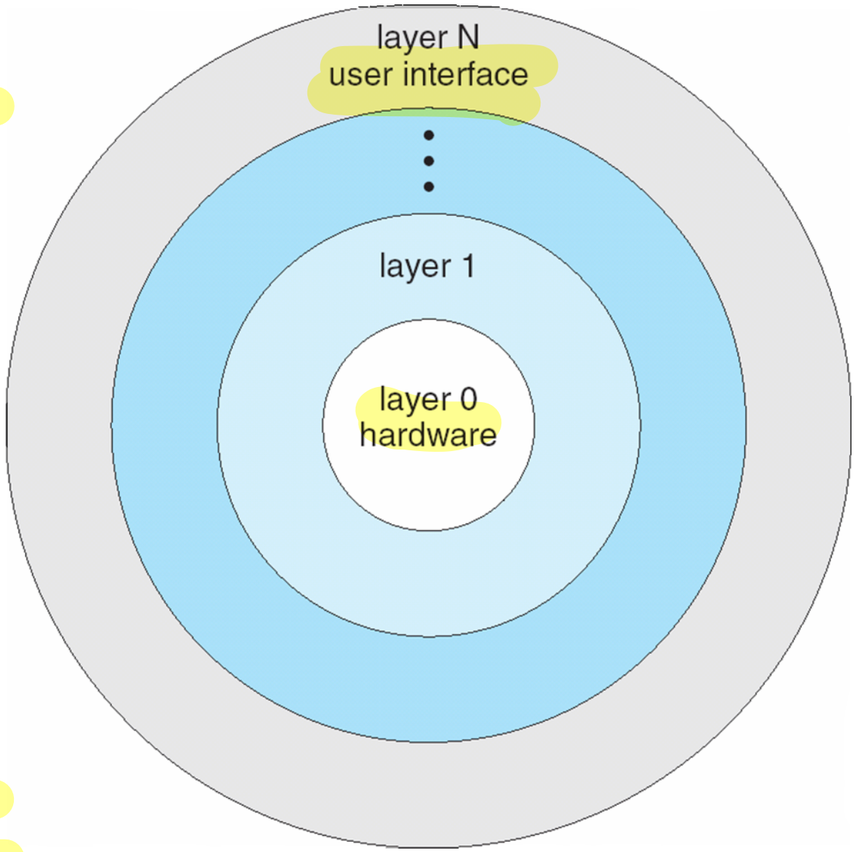
西南交通大学 Southwest Jiaotong University

Beyond simple but not fully layered



Layered Approach

- The operating system is divided into a number of layers (levels),
- each built on top of lower layers.
- The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- each uses functions (operations) and services of only lower-level layers



Microkernel System Structure

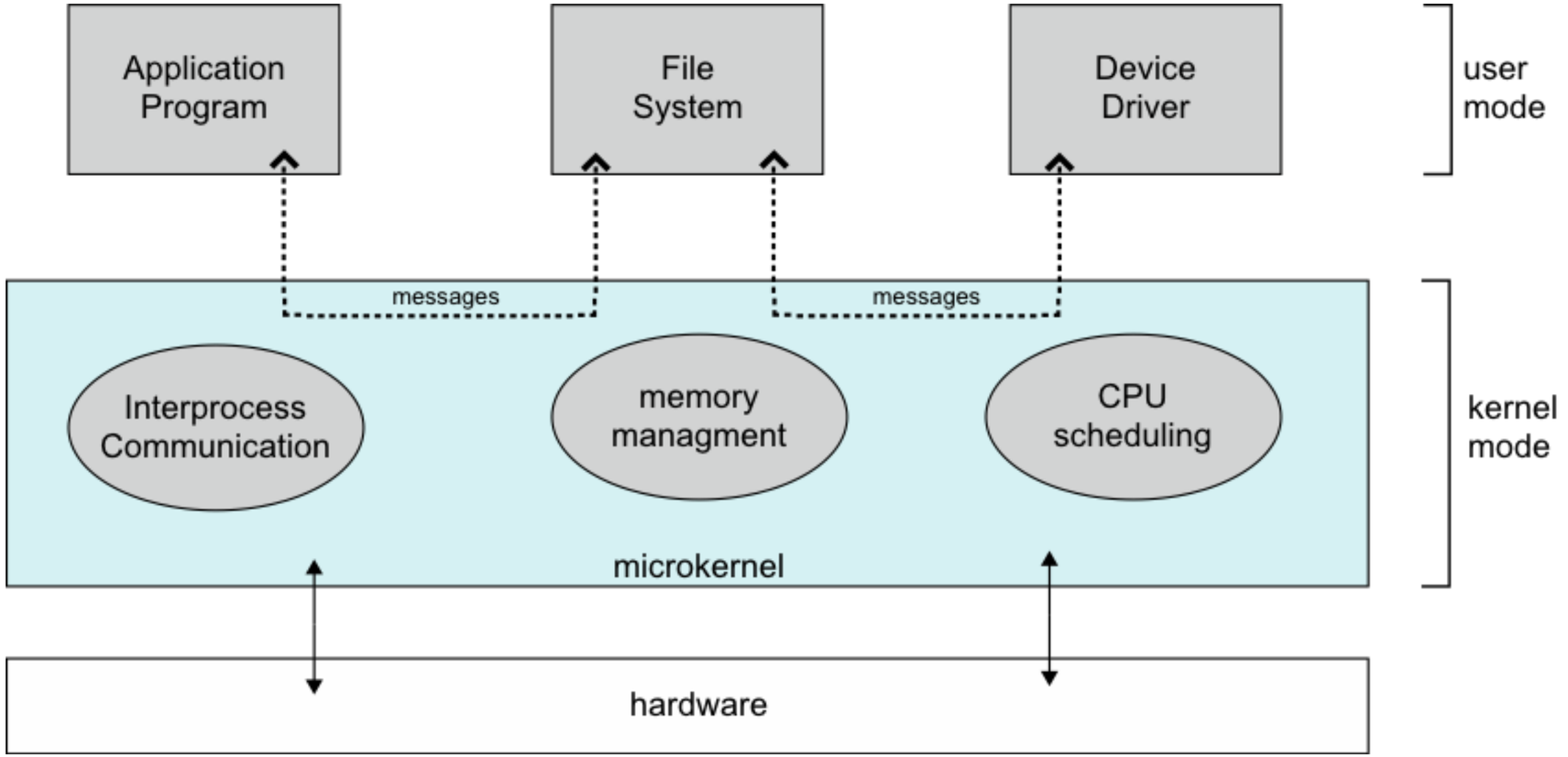
西南交通大学
Southwest Jiaotong University

- **Microkernel moves as much from the kernel into user space**
  - address spaces
  - interprocess communication (IPC)
  - basic scheduling
- **Benefits:**
  - Easier to extend a microkernel
  - Easier to port the operating system to new architectures
  - More reliable (less code is running in kernel mode)
  - More secure
- **Detriments:**
  - Performance overhead of user space to kernel space communication

开销

Microkernel System Structure

西南交通大学 Southwest Jiaotong University



Communication takes place between user modules using message passing

Modules

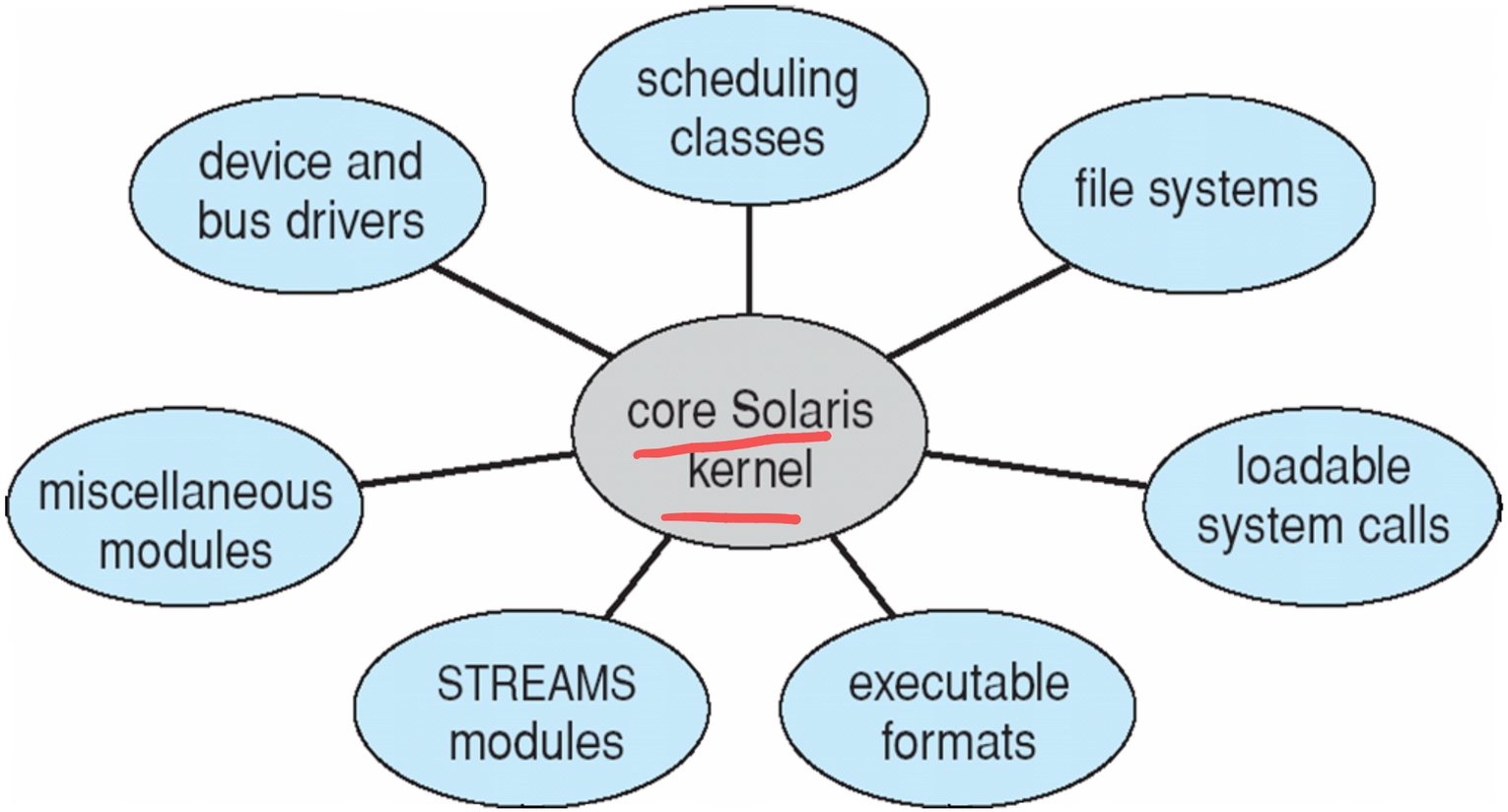
西南交通大学
Southwest Jiaotong University

- **Most modern operating systems implement loadable kernel modules**
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to the others over known interfaces
  - Each is loadable as needed within the kernel
- **Overall, similar to layers but with more flexible**
  - Linux, Solaris, etc

可加载的内核模块

Solaris Modular Approach

西南交通大学
Southwest Jiaotong University



In summary

- **Operating System Objectives**
  - Convenience
  - Efficiency
  - Ability to evolve
- **Evolution of Operating System**
  - Batch, multiprogramming, time-sharing, real-time, distributed…
- **Operating System Structures**
  - Services, System call, kernel…